

Intelligent Industrial Robotics Security Platform

OT/IT Convergence • Safety-System Protection • AI-Driven Anomaly Detection
End-to-End Architecture & Working Explanation

A complete walkthrough of what the system is, how every part works, how data and control flow across the OT/IT trust boundaries, and how it maps 100% to the Topic 114 project requirements.
Written so you can explain the project end to end to anyone.

Smart-manufacturing reference design • IEC 62443 / NIST SP 800-82 aligned

Contents

- 1.** What this project is (in one minute)
- 2.** The big picture: 5 zones, 1 mission
- 3.** System architecture, end to end
- 4.** Zone-by-zone walkthrough
- 5.** Network design & segmentation
- 6.** How a detection works (data flow)
- 7.** The AI: anomaly detection explained simply
- 8.** Safety system & functional safety (the E-stop)
- 9.** Incident response & recovery workflow
- 10.** Vulnerability management
- 11.** DevSecOps pipeline for industrial code
- 12.** Security architecture (defense in depth)
- 13.** Deployment: on-prem, hybrid, cloud
- 14.** 100% requirements traceability
- 15.** Security hardening applied (what was fixed)
- 16.** Technology stack
- 17.** Glossary for non-experts

1. What this project is (in one minute)

This project is a **security platform that protects an industrial robot** on a smart-manufacturing line. A factory robot is controlled by a PLC (a small industrial computer) that speaks **Modbus/TCP** and **ROS2** (the Robot Operating System). Those industrial protocols were designed for reliability, not security - they have little or no authentication - so if an attacker reaches the factory network they can send malicious commands that could damage equipment or hurt a worker.

The platform wraps that robot cell in a complete defensive system that does five things at once:

- **Segments the network** into OT/IT/DMZ/management zones so a breach in the office network cannot reach the plant floor (OT/IT convergence security).
- **Watches all industrial traffic** passively with intrusion-detection sensors and parses the OT protocols (Zeek, Suricata, ntopng).
- **Uses AI** to learn what normal robot/PLC behaviour looks like and flag anomalies that signal a cyber-physical attack (TensorFlow + scikit-learn).
- **Protects the safety system** - emergency stop, interlocks and a safety PLC - and can trip an authenticated E-stop when an attack is detected (functional safety / IEC 62443).
- **Responds automatically** with incident-response playbooks, captures forensics, manages vulnerabilities, and validates all PLC/robot code through a DevSecOps pipeline.

It is built from open-source, cloud-native tools and is delivered as a set of Docker containers that emulate a real plant on a single host - so the whole OT/IT environment can be demonstrated end to end.

2. The big picture: five zones, one mission

Everything is organised using the **Purdue model** - the standard way industrial networks are layered. Think of it as concentric rings of trust: the robot and its controller sit in the most protected ring, and every outer ring is separated by a checkpoint. The platform implements five logical zones:

- **OT zone** (operational technology): the robot, the production PLC, and the safety system. This is the crown jewels - it has no direct internet access.
- **SEC zone** (security monitoring): passive sensors that only *watch* a mirror of OT traffic. They can never inject packets into OT.
- **AI zone** (analytics & response): the machine-learning detector, the incident-response engine, and the monitoring stack (Prometheus/Grafana).
- **IT zone**: ordinary corporate-style services - here, the Gitea code repository and CI/CD runner.
- **DMZ** (demilitarised zone): the only place OT and IT are allowed to meet, through brokered services - a Guacamole remote-access jump host and a read-only historian.
- **MGMT zone**: the operator's plane - the web dashboard used to watch and control the cell.

The mission of every zone boundary is the same: **let the right data through, and stop everything else**. That is the heart of OT/IT convergence security.

3. System architecture, end to end

The diagram below shows every major service and how the zones connect. Follow the two journeys: the **monitoring journey** (robot traffic flows out to the AI to be analysed) and the **control journey** (an operator's command flows in to the PLC). Notice the AI zone never sits on the OT data path - it receives a copy of traffic, keeping the plant isolated.

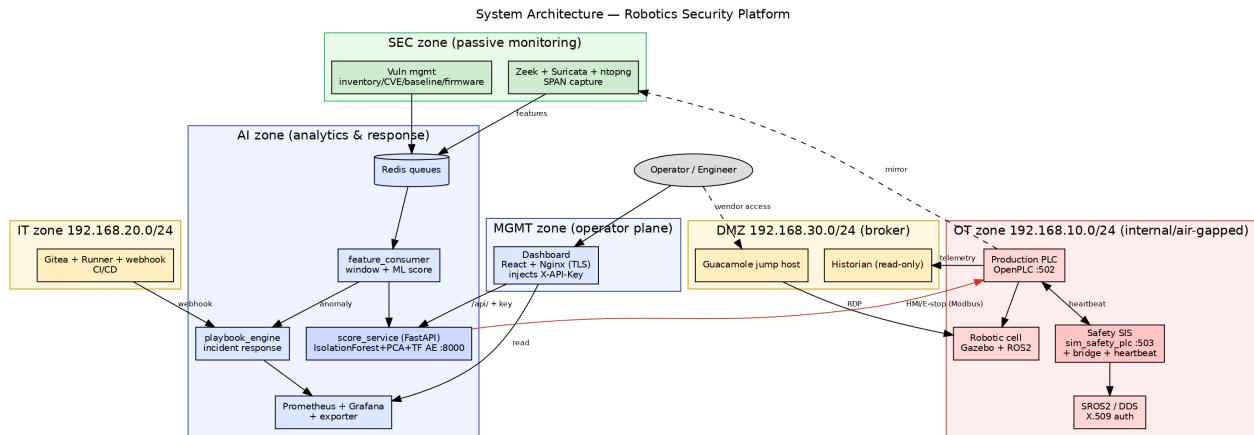


Figure 1 - System architecture: services grouped by trust zone, with monitoring and control flows.

Why it is laid out this way: the robot and safety controller are placed in the internal OT zone because they are real-time and safety-critical; the IDS sensors are passive and therefore live in their own SEC zone; the heavy AI and dashboards are kept separate so a problem there can never stall the plant; and the DMZ is the single controlled meeting point between plant and office.

4. Zone-by-zone walkthrough

Here is what actually runs inside each container and why.

4.1 OT zone (container-ot)

Runs the simulated factory cell: a **Gazebo** 3D robot arm driven by a ROS2 motion node, the **OpenPLC** production controller executing ladder logic (production.st) over Modbus on port 502, and the **safety system** on port 503. The safety system is three cooperating processes: a safety PLC datastore, a safety bridge that links Modbus to ROS2, and a heartbeat publisher. ROS2 messages are protected by **SROS2** (DDS security) which requires every participant to present an X.509 certificate.

4.2 SEC zone (container-sec)

Passive network security monitoring. **Zeek** and **Suricata** parse industrial protocols (Modbus, DNP3, OPC-UA) from a mirrored copy of OT traffic; **ntopng** gives live flow visibility. A feature_pusher turns parsed traffic into numerical feature rows and ships them to the AI zone. This zone also runs the vulnerability-management tools (asset inventory, CVE correlation, configuration-baseline and integrity checks, firmware workflow).

4.3 AI zone (container-ai)

The brain. **Redis** buffers feature and event queues; feature_consumer aggregates traffic into 5-second windows and scores each with three ML models; score_service is a FastAPI application exposing the models plus the SCADA/HMI control API; alert_bridge records anomalies; playbook_engine runs incident response; and **Prometheus + Grafana** provide metrics and dashboards. A webhook receiver triggers the DevSecOps pipeline.

4.4 IT zone, DMZ and MGMT

- **IT:** a **Gitea** git server and CI runner hold the robot/PLC source code and run the security pipeline on every change.
- **DMZ:** **Apache Guacamole** is a browser-based jump host that brokers vendor remote access into OT without exposing the plant directly; a read-only historian replica serves plant data to the IT side one-way.
- **MGMT:** the **React dashboard** behind Nginx. Nginx injects the API key server-side, so the browser never holds the secret while still being able to monitor and control the cell.

5. Network design & segmentation

Each zone is a separate virtual network with its own subnet. The OT network is declared `internal`, which means the operating system gives it **no route to the internet** - the strongest isolation control in the stack. A host firewall additionally blocks any direct IT-to-OT traffic, forcing everything through the DMZ.

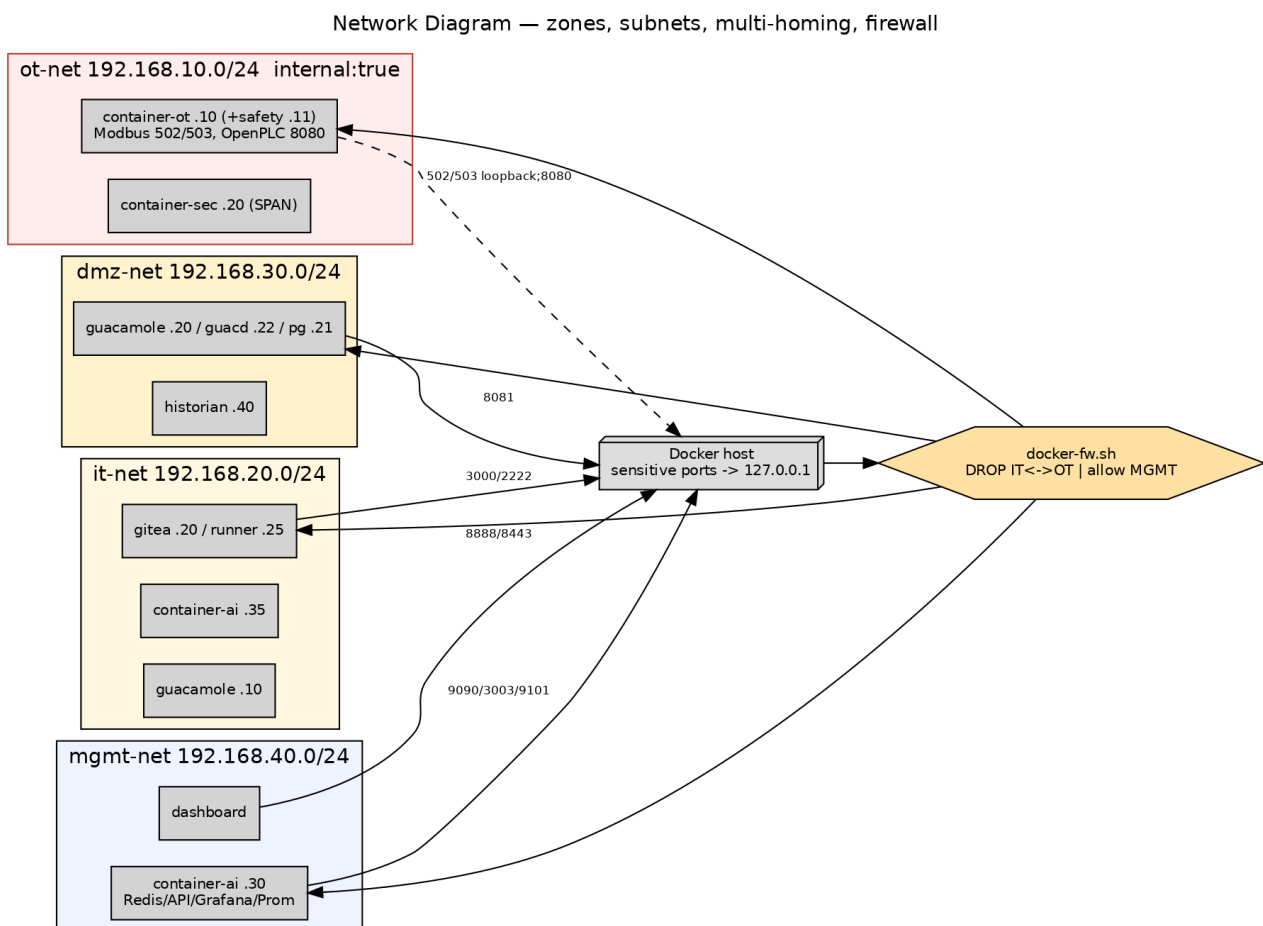


Figure 2 - Network diagram: four subnets, container addresses, host firewall, and loopback-bound sensitive ports.

Some containers are intentionally connected to more than one network (multi-homed) so that a single host can emulate cross-zone services. After the security hardening, the unauthenticated industrial ports (Modbus 502/503) and the raw scoring/CI ports (8000/9000) are bound to the host loopback only, so they are never reachable from the wider LAN; the operator dashboards remain published.

6. How a detection works (data flow)

This is the most important flow to understand. When traffic crosses the robot network, the platform turns it into a security decision in about five seconds:

1. **Capture:** OT traffic is mirrored to the SEC zone (the plant copy keeps running untouched).
2. **Parse:** Zeek/Suricata decode the Modbus/ROS2 conversation - who talked to whom, which function codes, how many writes, to which registers.
3. **Featurise:** feature_pusher converts each conversation into a 20-number feature vector and pushes it to Redis.
4. **Window & score:** feature_consumer groups rows into 5-second windows and scores each window with three ML models.
5. **Decide:** if any model's score crosses its calibrated threshold, the window is an anomaly.
6. **Alert:** alert_bridge writes a structured alert; a metric is exposed to Prometheus; the dashboard lights up.
7. **Respond:** the playbook engine opens an incident and begins containment (next section).

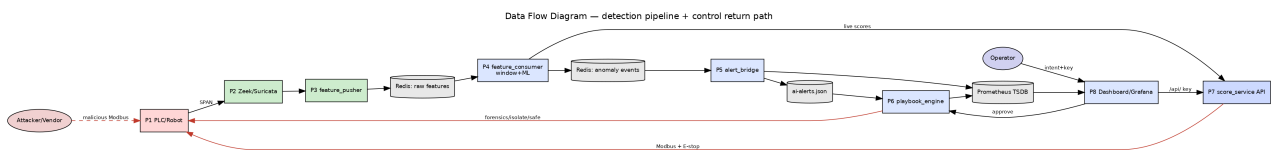


Figure 3 - Data-flow diagram: detection pipeline (left to right) and the operator control return path.

7. The AI: anomaly detection explained simply

The platform does **not** use a list of known-attack signatures alone. Instead it learns a model of **normal** behaviour and flags anything that deviates - this is how it catches novel cyber-physical attacks. Three complementary models vote:

- **Isolation Forest** (scikit-learn): a tree-based outlier detector. It is fast and great at spotting rare combinations of feature values, e.g. a sudden burst of writes from an address that normally only reads.
- **PCA autoencoder**: compresses each window and measures the reconstruction error. Normal traffic reconstructs cleanly; anomalous traffic does not.
- **TensorFlow dense autoencoder**: a neural-network version of the same idea, included to meet the deep-learning requirement and to catch subtler non-linear patterns.

The 20 features describe the *shape* of the traffic: message and write/read counts, number of unique function codes and addresses, mean/spread of register addresses and quantities, exception rate, function-code entropy, inter-arrival timing, and - crucially for OT - whether writes originate from outside the OT zone. Models are trained on **pure normal traffic only** (no attack contamination) so they do not learn to tolerate attacks and do not raise false alarms on healthy operation.

On top of raw scoring, the service computes a **trend** (rising/stable/falling) and a simple linear **breach prediction**, giving operators predictive analytics rather than just point alerts.

8. Safety system & functional safety (the E-stop)

In OT, **safety outranks security**: the system must protect people and machinery even while under attack. The platform models a **Safety Instrumented System (SIS)** separate from the production PLC.

- The production PLC sends a **1 Hz heartbeat** to the safety controller. If the heartbeat stops, a **watchdog** trips the cell into a safe state.
- An **emergency stop (E-stop)** can be asserted: the dashboard writes the E-stop coil on the production PLC and signals the safety controller. Once tripped, the EMERGENCY state is **latched**.
- Clearing the E-stop is always a **deliberate operator action** (a reset command) - never automatic - which is the correct functional-safety behaviour and is why the E-stop is preserved as a first-class feature.
- ROS2 safety topics (/safety/state, /safety/request) are carried over **SROS2**, so only certificate-holding nodes can participate.

This directly implements the requirement to deploy security controls for safety-critical systems (emergency stops, interlocks, safety PLCs) and to validate SIS integrity.

9. Incident response & recovery workflow

When an alert fires, the `playbook_engine` turns it into a managed incident. Playbooks are simple Markdown files with a machine-readable header that maps an alert category to a list of response steps. Steps are either automatic or require human approval:

- **Automatic steps** run immediately - e.g. capture forensic evidence, isolate the offending source with a firewall rule.
- **Approval steps** (e.g. assert a safe state / E-stop) pause and wait for an operator to approve on the dashboard, because they change the physical process.
- The engine de-duplicates a single attack that trips several detectors into one **campaign**, and records every action to an incident log for audit.

Incident Workflow — attack -> detect -> respond -> recover

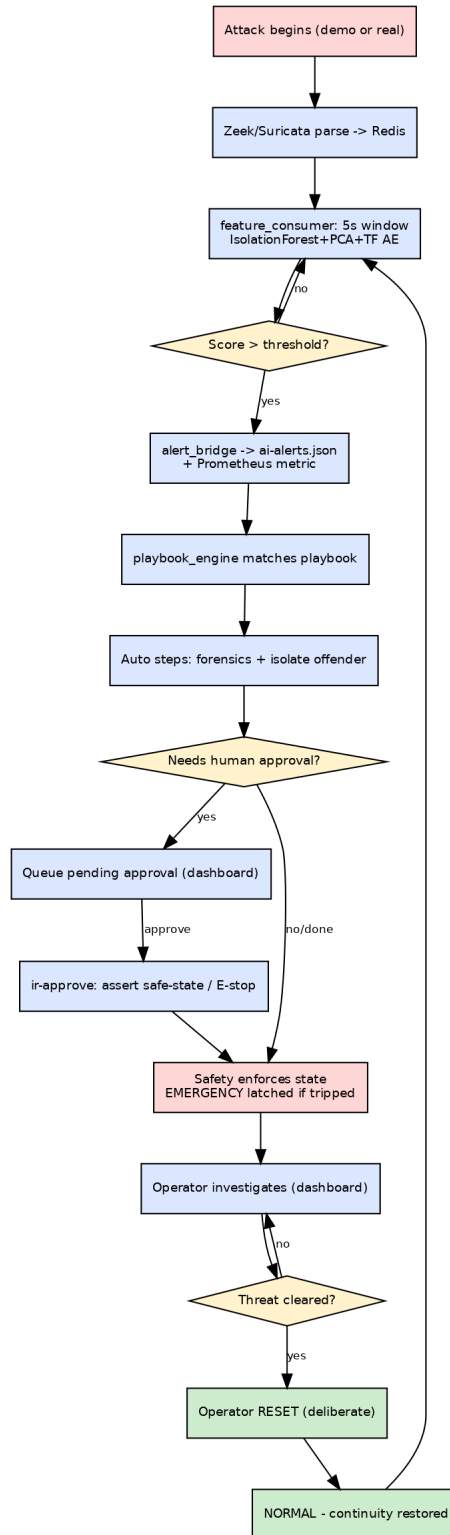


Figure 4 - Incident workflow: attack to detection to automated containment to human-approved safe-state to deliberate recovery.

Recovery is built to prioritise **manufacturing continuity and safety**: the operator investigates on the dashboard and, once the threat is cleared, issues a deliberate reset that returns the cell to normal.

10. Vulnerability management

The SEC zone continuously answers 'what do we have, and what is wrong with it?'

- **Asset inventory** (Nmap-based): discovers devices, open ports and services on the OT network.
- **CVE correlation**: matches discovered software/firmware versions against a CVE database to list known vulnerabilities.
- **Configuration baseline**: compares live configuration to an approved IEC-62443 security baseline and reports drift.
- **Integrity baseline**: hashes critical files (e.g. the PLC program) so any tampering is detected.
- **Firmware workflow**: a controlled process for validating and staging firmware/patch updates for robotics platforms.

11. DevSecOps pipeline for industrial code

PLC ladder logic, HMI screens and robot configuration are **code**, and bad code on a robot is dangerous. Every change pushed to Gitea triggers (via an HMAC-signed webhook) a six-gate pipeline:

- **PLC lint** - checks the Structured Text safety logic.
- **HMI lint** - checks HMI/SCADA screen definitions.
- **SROS2 lint** - checks the ROS2 security permission files.
- **Vulnerability gate** - fails the build on critical findings.
- **Baseline gate** - fails on critical configuration drift.
- **Acceptance** - replays normal/attack traffic and exercises the safety loop.

On a full pass the pipeline produces a hashed, signed artifact bundle, giving traceable, secure delivery of industrial automation code.

12. Security architecture (defense in depth)

No single control is trusted on its own. An attacker has to defeat every layer in turn, while defenders get telemetry flowing the other way.

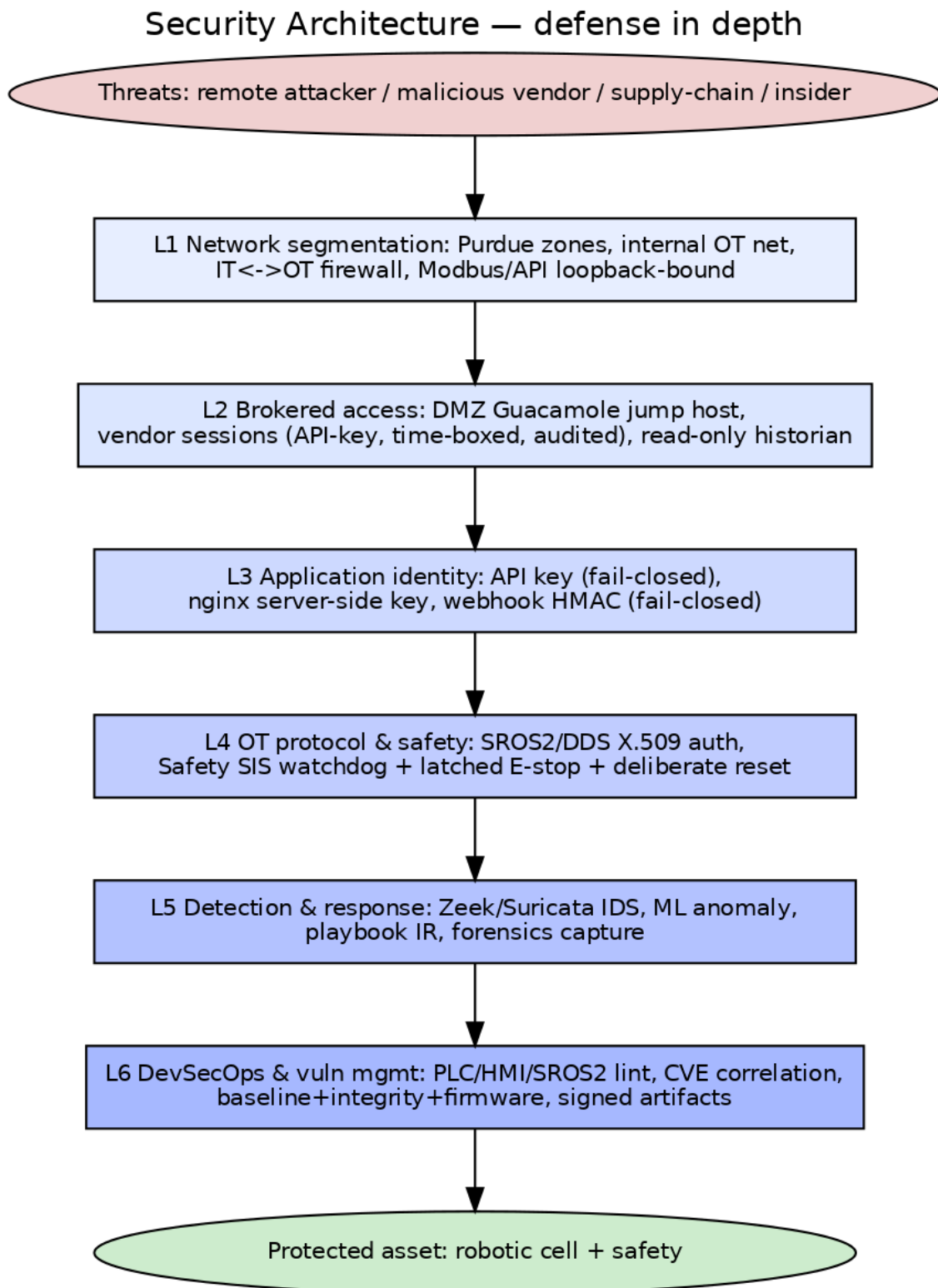


Figure 5 - Security architecture: six complementary control layers between threats and the robot.

Layers one to four keep attackers out (segmentation, brokered access, application identity, OT/safety authentication); layer five detects and responds to whatever slips through; layer six secures the code and configuration supply chain. Together they implement IEC 62443 zones-and-conduits and NIST SP 800-82.

13. Deployment: on-prem, hybrid, cloud

Today the whole platform runs **on-prem at the edge** - one Docker host represents one plant, which matches how real OT is deployed (close to the machinery, able to survive a network outage). The design also shows a **target hybrid** model.

Cloud / Hybrid / On-Prem — safety stays at the edge

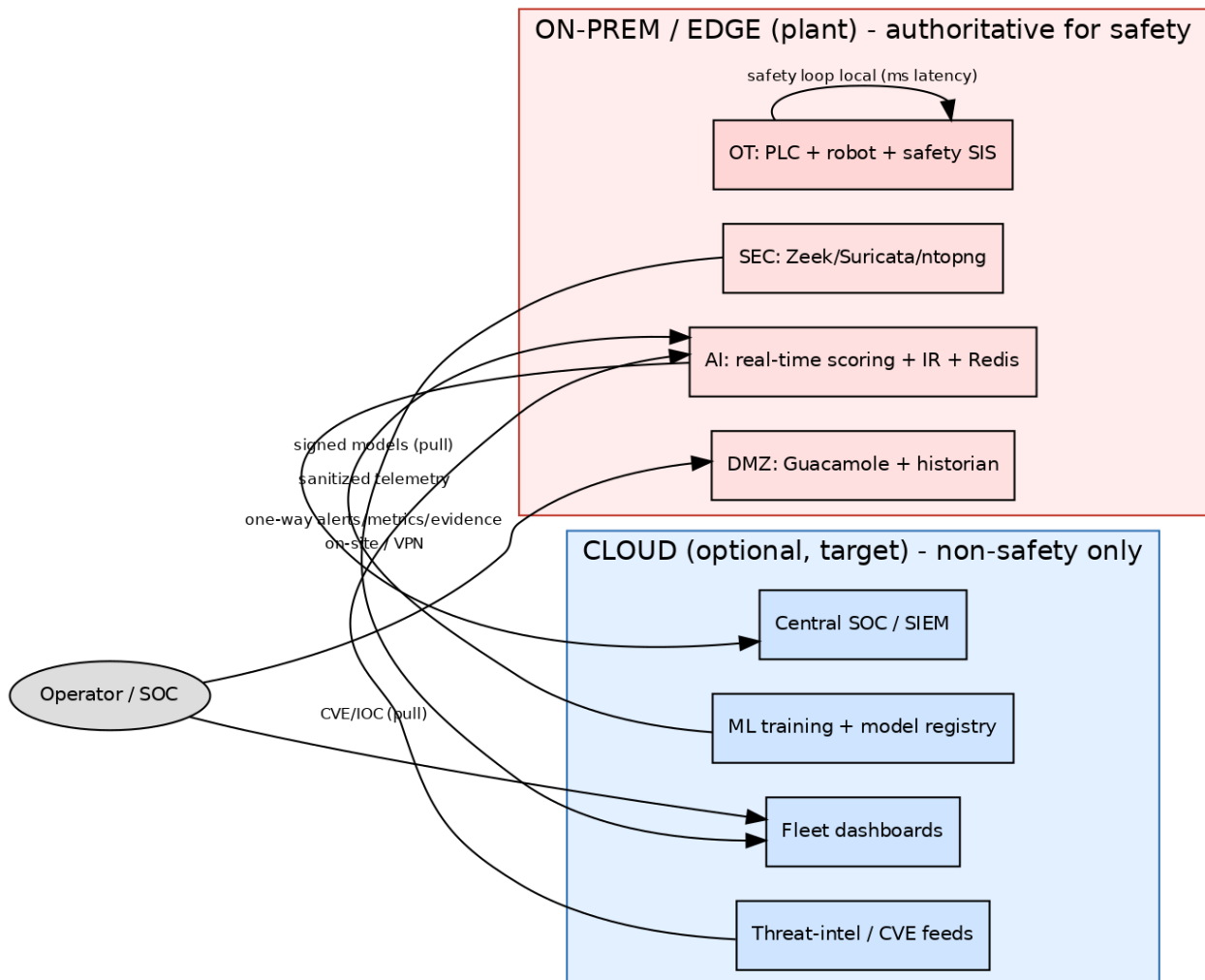


Figure 6 - On-prem/edge today, with an optional cloud tier for fleet SOC, model training and dashboards.

The golden rule: **anything safety- or latency-critical stays at the edge**. The cloud only ever receives derived, non-safety data (alerts, metrics, evidence) one-way, and only sends back pull-based, signed artifacts (trained models, threat-intel). No cloud service can ever command the plant - so losing the cloud never stops production or safety.

14. 100% requirements traceability

Every objective in the Topic 114 requirements maps to a concrete component in this codebase:

Requirement (Topic 114)	Where it is implemented
Secure OT/IT architecture, industrial DMZ, microsegmentation	docker-compose 4 networks + internal OT net; infra/docker-fw.sh; DMZ Guacamole
Secure remote / vendor access management	vendor_access.py + Apache Guacamole jump host (DMZ)
Network traffic analysis & industrial protocol monitoring	vm-sec Zeek + Suricata + ntopng; modbus/dnp3/opcua parsers

Requirement (Topic 114)	Where it is implemented
ML models for robotic behavioural anomaly detection	train_iforest / train_autoencoder / train_autoencoder_tf; feature_consumer; score_service
Predictive security analytics for cyber-physical attacks	score_service trend + breach prediction (/api/trend)
Automated threat response + safety integration	ir/playbook_engine.py + playbooks; E-stop via score_service
Controls for safety systems (E-stop, interlocks, safety PLC)	sim_safety_plc + safety_bridge + safety_heartbeat; production.st
IEC 62443 compliance automation	vm-sec/vuln/baseline_check.py + baselines; sros2/governance
Safety Instrumented System (SIS) security & integrity	safety supervisor logic; integrity_baseline.py
Automated vulnerability scanning for robots/control systems	vm-sec/vuln/inventory.py (Nmap) + cve_correlate.py
Secure firmware management & patch deployment	vm-sec/vuln/firmware_workflow.py
Configuration management & baseline enforcement	baseline_check.py + integrity_baseline.py + baselines/*.yaml
Security validation for PLC programming	devsecops/plc_lint.py (pipeline Gate 1)
Secure development for HMI/SCADA	devsecops/hmi_lint.py (Gate 2); score_service HMI API
Automated security testing for automation workflows	run_pipeline.sh 6 gates; webhook_receiver.py
Automated incident detection & response	playbook_engine.py + alert_bridge.py
Recovery prioritising continuity & safety	approval-gated playbooks; deliberate E-stop reset
Forensic analysis for industrial incidents	ir/forensics_capture.sh + evidence store
Tools: Zeek, Suricata, ntopng, TF, sklearn, Prometheus, Grafana, Nmap, SROS2	present across vm-sec / vm-ai / vm-ot

15. Security hardening applied (what was fixed)

A security audit of the platform identified several issues; the following fixes were applied **without removing any feature, the E-stop, or the live attack demo**:

- **Authentication is now fail-closed**: the scoring API and the CI webhook reject requests when no key/secret is configured (previously they silently allowed everything).
- **Previously-open endpoints are protected**: vendor remote-access provisioning, incident data, the security-report feed, PLC state and log endpoints now require the API key (the dashboard is unaffected because Nginx supplies the key).
- **Secrets rotated and de-duplicated**: the API key and webhook secret are now different values; the Grafana admin password is no longer 'admin'.
- **Industrial ports taken off the LAN**: Modbus (502/503) and the raw API/webhook (8000/9000) are bound to host loopback only.
- **Resilience**: core services are now restart-supervised; the boot-time state wipe is optional (set LAB_RESET_STATE=0 to keep history); the container entrypoint was de-corrupted.
- **Hygiene**: the Redis health-check no longer leaks the password on the process list.

The E-stop, safety logic, attack-injection demo and live telemetry were intentionally left fully functional so the platform still demonstrates end to end.

16. Technology stack

Area	Tools
OT security / IDS	Zeek, Suricata, ntopng, custom Modbus/DNP3/OPC-UA parsers
ML / analytics	TensorFlow (CPU), scikit-learn, NumPy, pandas
Robotics / OT	ROS2 Humble, Gazebo, OpenPLC (IEC 61131-3 ST), SROS2 / Cyclone DDS, pymodbus
API / data plane	FastAPI, Uvicorn, Redis
Monitoring	Prometheus, Grafana, custom exporter
Vuln scanning	Nmap, custom OT scanners, CVE database
DevSecOps / IT	Gitea + Act runner, HMAC webhook, GPG-signed artifacts
Remote access / DMZ	Apache Guacamole, PostgreSQL, read-only historian
Frontend	React + TypeScript + Vite, Nginx (TLS reverse proxy)
Packaging	Docker + Docker Compose (4 segmented networks)
Standards	IEC 62443 / ISA-62443, NIST SP 800-82

17. Glossary for non-experts

OT / IT - Operational Technology is the tech that runs physical machinery; IT is normal computing. 'Convergence' is connecting them safely.

PLC - Programmable Logic Controller - the rugged computer that controls the robot/line.

Modbus / DNP3 / OPC-UA - Common industrial communication protocols; mostly unauthenticated by design.

ROS2 / SROS2 - Robot Operating System 2 and its security layer (certificate-based authentication for robot messages).

Purdue model - The standard layered reference model for industrial networks (zones from plant floor up to enterprise).

DMZ - A buffer network where two zones of different trust meet through controlled, brokered services.

IDS - Intrusion Detection System - sensors that watch traffic for malicious or abnormal activity.

Anomaly detection - Flagging behaviour that differs from a learned 'normal', rather than matching known-attack signatures.

E-stop / SIS - Emergency stop and the independent Safety Instrumented System that forces the machine into a safe state.

IEC 62443 / NIST 800-82 - The leading international standards for industrial / OT cybersecurity.

DevSecOps - Building security testing directly into the software delivery pipeline.

Prepared as an end-to-end explanation companion to ARCHITECTURE-DIAGRAMS.md and INTERVIEW-DEMO-GUIDE.md. Diagrams in this document mirror the Mermaid sources in the diagram pack.